Extensions

The behavior of some of the readers and writers can be adjusted by enabling or disabling various extensions.

An extension can be enabled by adding `+EXTENSION` to the format name and disabled by adding `-EXTENSION`. For example, `--from markdown_strict+footnotes` is strict Markdown with footnotes enabled, while `--from markdown-footnotes-pipe_tables` is pandoc's Markdown without footnotes or pipe tables.

The Markdown reader and writer make by far the most use of extensions. Extensions only used by them are therefore covered in the section Pandoc's Markdown below (see Markdown variants for `commonmark` and `gfm`). In the following, extensions that also work for other formats are covered.

Note that Markdown extensions added to the `ipynb` format affect Markdown cells in Jupyter notebooks (as do command-line options like `--markdown-headings`).

Typography

Extension: `smart`

Interpret straight quotes as curly quotes, `---` as em-dashes, `--` as en-dashes, and `...` as ellipses. Nonbreaking spaces are inserted after certain abbreviations, such as "Mr."

This extension can be enabled/disabled for the following formats:

input formats `markdown`, `commonmark`, `latex`, `mediawiki`, `org`, `rst`, `twiki`, `html`
output formats `markdown`, `latex`, `context`, `org`, `rst`
enabled by default in `markdown`, `latex`, `context` (both input and output)

Note: If you are *writing* Markdown, then the `smart` extension has the reverse effect: what would have been curly quotes comes out straight.

In LaTeX, `smart` means to use the standard TeX ligatures for quotation marks (`` ` `` and `''` for double quotes, `` ` `` and `'` for single quotes) and dashes (`--` for en-dash and `---` for em-dash). If `smart` is disabled, then in reading LaTeX pandoc will parse these characters literally. In writing LaTeX, enabling `smart` tells pandoc to use the ligatures when possible; if `smart` is disabled pandoc will use unicode quotation mark and dash characters.

Headings and sections

Extension: `auto_identifiers`

A heading without an explicitly specified identifier will be automatically assigned a unique identifier based on the heading text.

This extension can be enabled/disabled for the following formats:

input formats `markdown`, `latex`, `rst`, `mediawiki`, `textile`
output formats `markdown`, `muse`
enabled by default in `markdown`, `muse`

The default algorithm used to derive the identifier from the heading text is:

- Remove all formatting, links, etc.
- Remove all footnotes.
- Remove all non-alphanumeric characters, except underscores, hyphens, and periods.
- Replace all spaces and newlines with hyphens.
- Convert all alphabetic characters to lowercase.
- Remove everything up to the first letter (identifiers may not begin with a number or punctuation mark).
- If nothing is left after this, use the identifier `section`.

Thus, for example,

Heading	Identifier
`Heading identifiers in HTML`	`heading-identifiers-in-html`
`Maître d'hôtel`	`maître-dhôtel`
`*Dogs*?--in *my* house?`	`dogs--in-my-house`
`[HTML], [S5], or [RTF]?`	`html-s5-or-rtf`
`3. Applications`	`applications`
`33`	`section`

These rules should, in most cases, allow one to determine the identifier from the heading text. The exception is when several headings have the same text; in this case, the first will get an identifier as described above; the second will get the same identifier with `-1` appended; the third with `-2`; and so on.

(However, a different algorithm is used if `gfm_auto_identifiers` is enabled; see below.)

These identifiers are used to provide link targets in the table of contents generated by the `--toc|--table-of-contents` option. They also make it easy to provide links from one section of a document to another. A link to this section, for example, might look like this:

```
See the section on
[heading identifiers](#heading-identifiers-in-html-latex-and-context).
```

Note, however, that this method of providing links to sections works only in HTML, LaTeX, and ConTeXt formats.

If the `--section-divs` option is specified, then each section will be wrapped in a `section` (or a `div`, if `html4` was specified), and the identifier will be attached to the enclosing `<section>` (or `<div>`) tag rather than the heading itself. This allows entire sections to be manipulated using JavaScript or treated differently in CSS.

Extension: `ascii_identifiers`

Causes the identifiers produced by `auto_identifiers` to be pure ASCII. Accents are stripped off of accented Latin letters, and non-Latin letters are omitted.

Extension: `gfm_auto_identifiers`

Changes the algorithm used by `auto_identifiers` to conform to GitHub's method. Spaces are converted to dashes (-), uppercase characters to lowercase characters, and punctuation characters other than - and _ are removed. Emojis are replaced by their names.

Math Input

The extensions `tex_math_dollars`, `tex_math_gfm`, `tex_math_single_backslash`, and `tex_math_double_backslash` are described in the section about Pandoc's Markdown.

However, they can also be used with HTML input. This is handy for reading web pages formatted using MathJax, for example.

Raw HTML/TeX

The following extensions are described in more detail in their respective sections of Pandoc's Markdown:

- `raw_html` allows HTML elements which are not representable in pandoc's AST to be parsed as raw HTML. By default, this is disabled for HTML input.
- `raw_tex` allows raw LaTeX, TeX, and ConTeXt to be included in a document. This extension can be enabled/disabled for the following formats (in addition to `markdown`):

  input formats `latex`, `textile`, `html` (environments, `\ref`, and `\eqref` only), `ipynb`

output formats `textile`, `commonmark`

Note: as applied to `ipynb`, `raw_html` and `raw_tex` affect not only raw TeX in Markdown cells, but data with mime type `text/html` in output cells. Since the `ipynb` reader attempts to preserve the richest possible outputs when several options are given, you will get best results if you disable `raw_html` and `raw_tex` when converting to formats like `docx` which don't allow raw `html` or `tex`.

- `native_divs` causes HTML `div` elements to be parsed as native pandoc Div blocks. If you want them to be parsed as raw HTML, use `-f html-native_divs+raw_html`.
- `native_spans` causes HTML `span` elements to be parsed as native pandoc Span inlines. If you want them to be parsed as raw HTML, use `-f html-native_spans+raw_html`. If you want to drop all `div`s and `span`s when converting HTML to Markdown, you can use `pandoc -f html-native_divs-native_spans -t markdown`.

Literate Haskell support

Extension: `literate_haskell`

Treat the document as literate Haskell source.

This extension can be enabled/disabled for the following formats:

input formats `markdown`, `rst`, `latex`
output formats `markdown`, `rst`, `latex`, `html`

If you append `+lhs` (or `+literate_haskell`) to one of the formats above, pandoc will treat the document as literate Haskell source. This means that

- In Markdown input, "bird track" sections will be parsed as Haskell code rather than block quotations. Text between `\begin{code}` and `\end{code}` will also be treated as Haskell code. For ATX-style headings the character '=' will be used instead of '#'.
- In Markdown output, code blocks with classes `haskell` and `literate` will be rendered using bird tracks, and block quotations will be indented one space, so they will not be treated as Haskell code. In addition, headings will be rendered setext-style (with underlines) rather than ATX-style (with '#' characters). (This is because ghc treats '#' characters in column 1 as introducing line numbers.)
- In restructured text input, "bird track" sections will be parsed as Haskell code.
- In restructured text output, code blocks with class `haskell` will be rendered using bird tracks.
- In LaTeX input, text in `code` environments will be parsed as Haskell code.

- In LaTeX output, code blocks with class `haskell` will be rendered inside `code` environments.
- In HTML output, code blocks with class `haskell` will be rendered with class `literatehaskell` and bird tracks.

Examples:

```
pandoc -f markdown+lhs -t html
```

reads literate Haskell source formatted with Markdown conventions and writes ordinary HTML (without bird tracks).

```
pandoc -f markdown+lhs -t html+lhs
```

writes HTML with the Haskell code in bird tracks, so it can be copied and pasted as literate Haskell source.

Note that GHC expects the bird tracks in the first column, so indented literate code blocks (e.g. inside an itemized environment) will not be picked up by the Haskell compiler.

Other extensions

Extension: `empty_paragraphs`

Allows empty paragraphs. By default empty paragraphs are omitted.

This extension can be enabled/disabled for the following formats:

input formats `docx`, `html`
output formats `docx`, `odt`, `opendocument`, `html`, `latex`

Extension: `native_numbering`

Enables native numbering of figures and tables. Enumeration starts at 1.

This extension can be enabled/disabled for the following formats:

output formats `odt`, `opendocument`, `docx`

Extension: `xrefs_name`

Links to headings, figures and tables inside the document are substituted with cross-references that will use the name or caption of the referenced item. The original link text is replaced once the generated document is refreshed. This extension can be combined with `xrefs_number` in which case numbers will appear before the name.

Text in cross-references is only made consistent with the referenced item once the document has been refreshed.

This extension can be enabled/disabled for the following formats:

output formats `odt`, `opendocument`

Extension: `xrefs_number`

Links to headings, figures and tables inside the document are substituted with cross-references that will use the number of the referenced item. The original link text is discarded. This extension can be combined with `xrefs_name` in which case the name or caption numbers will appear after the number.

For the `xrefs_number` to be useful heading numbers must be enabled in the generated document, also table and figure captions must be enabled using for example the `native_numbering` extension.

Numbers in cross-references are only visible in the final document once it has been refreshed.

This extension can be enabled/disabled for the following formats:

output formats `odt`, `opendocument`

Extension: `styles`

When converting from docx, add `custom-styles` attributes for all docx styles, regardless of whether pandoc understands the meanings of these styles. Because attributes cannot be added directly to paragraphs or text in the pandoc AST, paragraph styles will cause Divs to be created and character styles will cause Spans to be created to hold the attributes. (Table styles will be added to the Table elements directly.) This extension can be used with docx custom styles.

input formats `docx`

Extension: `amuse`

In the `muse` input format, this enables Text::Amuse extensions to Emacs Muse markup.

Extension: `raw_markdown`

In the `ipynb` input format, this causes Markdown cells to be included as raw Markdown blocks (allowing lossless round-tripping) rather than being parsed. Use this only when you are targeting `ipynb` or a Markdown-based output format.

Extension: `citations` (typst)

When the `citations` extension is enabled in `typst` (as it is by default), `typst` citations will be parsed as native pandoc citations, and native pandoc citations will be rendered as `typst` citations.

Extension: `citations` (org)

When the `citations` extension is enabled in `org`, org-cite and org-ref style citations will be parsed as native pandoc citations, and org-cite citations will be used to render native pandoc citations.

Extension: `citations` (docx)

When `citations` is enabled in `docx`, citations inserted by Zotero or Mendeley or EndNote plugins will be parsed as native pandoc citations. (Otherwise, the formatted citations generated by the bibliographic software will be parsed as regular text.)

Extension: `fancy_lists` (org)

Some aspects of Pandoc's Markdown fancy lists are also accepted in `org` input, mimicking the option `org-list-allow-alphabetical` in Emacs. As in Org Mode, enabling this extension allows lowercase and uppercase alphabetical markers for ordered lists to be parsed in addition to arabic ones. Note that for Org, this does not include roman numerals or the `#` placeholder that are enabled by the extension in Pandoc's Markdown.

Extension: `element_citations`

In the `jats` output formats, this causes reference items to be replaced with `<element-citation>` elements. These elements are not influenced by CSL styles, but all information on the item is included in tags.

Extension: `ntb`

In the `context` output format this enables the use of Natural Tables (TABLE) instead of the default Extreme Tables (xtables). Natural tables allow more fine-grained global customization but come at a performance penalty compared to extreme tables.

Extension: `smart_quotes` (org)

Interpret straight quotes as curly quotes during parsing. When *writing* Org, then the `smart_quotes` extension has the reverse effect: what would have been curly quotes comes out straight.

This extension is implied if `smart` is enabled.

Extension: `special_strings` (org)

Interpret `---` as em-dashes, `--` as en-dashes, `\-` as shy hyphen, and `...` as ellipses.

This extension is implied if `smart` is enabled.

Extension: `tagging`

Enabling this extension with `context` output will produce markup suitable for the production of tagged PDFs. This includes additional markers for paragraphs and alternative markup for emphasized text. The `emphasis-command` template variable is set if the extension is enabled.

Pandoc's Markdown

Pandoc understands an extended and slightly revised version of John Gruber's Markdown syntax. This document explains the syntax, noting differences from original Markdown. Except where noted, these differences can be suppressed by using the `markdown_strict` format instead of `markdown`. Extensions can be enabled or disabled to specify the behavior more granularly. They are described in the following. See also Extensions above, for extensions that work also on other formats.

Philosophy

Markdown is designed to be easy to write, and, even more importantly, easy to read:

> A Markdown-formatted document should be publishable as-is, as plain text, without looking like it's been marked up with tags or formatting instructions.
> – John Gruber

This principle has guided pandoc's decisions in finding syntax for tables, footnotes, and other extensions.

There is, however, one respect in which pandoc's aims are different from the original aims of Markdown. Whereas Markdown was originally designed with HTML generation in mind, pandoc is designed for multiple output formats. Thus, while pandoc allows the embedding of raw HTML, it discourages it, and provides other, non-HTMLish ways of representing important document elements like definition lists, tables, mathematics, and footnotes.

Paragraphs

A paragraph is one or more lines of text followed by one or more blank lines. Newlines are treated as spaces, so you can reflow your paragraphs as you like. If you need a hard line break, put two or more spaces at the end of a line.

Extension: `escaped_line_breaks`

A backslash followed by a newline is also a hard line break. Note: in multiline and grid table cells, this is the only way to create a hard line break, since trailing spaces in the cells are ignored.

Headings

There are two kinds of headings: Setext and ATX.

Setext-style headings

A setext-style heading is a line of text "underlined" with a row of = signs (for a level-one heading) or – signs (for a level-two heading):

```
A level-one heading
===================

A level-two heading
-------------------
```

The heading text can contain inline formatting, such as emphasis (see Inline formatting, below).

ATX-style headings

An ATX-style heading consists of one to six # signs and a line of text, optionally followed by any number of # signs. The number of # signs at the beginning of the line is the heading level:

```
## A level-two heading

### A level-three heading ###
```

As with setext-style headings, the heading text can contain formatting:

```
# A level-one heading with a [link](/url) and *emphasis*
```

Extension: `blank_before_header`

Original Markdown syntax does not require a blank line before a heading. Pandoc does require this (except, of course, at the beginning of the document). The reason for the requirement is that it is all too easy for a `#` to end up at the beginning of a line by accident (perhaps through line wrapping). Consider, for example:

```
I like several of their flavors of ice cream:
#22, for example, and #5.
```

Extension: `space_in_atx_header`

Many Markdown implementations do not require a space between the opening `#`s of an ATX heading and the heading text, so that `#5 bolt` and `#hashtag` count as headings. With this extension, pandoc does require the space.

Heading identifiers

See also the `auto_identifiers` extension above.

Extension: `header_attributes`

Headings can be assigned attributes using this syntax at the end of the line containing the heading text:

```
{#identifier .class .class key=value key=value}
```

Thus, for example, the following headings will all be assigned the identifier `foo`:

```
# My heading {#foo}

## My heading ##    {#foo}

My other heading   {#foo}
---------------
```

(This syntax is compatible with PHP Markdown Extra.)

Note that although this syntax allows assignment of classes and key/value attributes, writers generally don't use all of this information. Identifiers, classes, and key/value attributes are

used in HTML and HTML-based formats such as EPUB and slidy. Identifiers are used for labels and link anchors in the LaTeX, ConTeXt, Textile, Jira markup, and AsciiDoc writers.

Headings with the class `unnumbered` will not be numbered, even if `--number-sections` is specified. A single hyphen (`-`) in an attribute context is equivalent to `.unnumbered`, and preferable in non-English documents. So,

```
# My heading {-}
```

is just the same as

```
# My heading {.unnumbered}
```

If the `unlisted` class is present in addition to `unnumbered`, the heading will not be included in a table of contents. (Currently this feature is only implemented for certain formats: those based on LaTeX and HTML, PowerPoint, and RTF.)

Extension: `implicit_header_references`

Pandoc behaves as if reference links have been defined for each heading. So, to link to a heading

```
# Heading identifiers in HTML
```

you can simply write

```
[Heading identifiers in HTML]
```

or

```
[Heading identifiers in HTML][]
```

or

```
[the section on heading identifiers][heading identifiers in
HTML]
```

instead of giving the identifier explicitly:

```
[Heading identifiers in HTML](#heading-identifiers-in-html)
```

If there are multiple headings with identical text, the corresponding reference will link to the first one only, and you will need to use explicit links to link to the others, as described above.

Like regular reference links, these references are case-insensitive.

Explicit link reference definitions always take priority over implicit heading references. So, in the following example, the link will point to `bar`, not to `#foo`:

```
# Foo

[foo]: bar

See [foo]
```

Block quotations

Markdown uses email conventions for quoting blocks of text. A block quotation is one or more paragraphs or other block elements (such as lists or headings), with each line preceded by a > character and an optional space. (The > need not start at the left margin, but it should not be indented more than three spaces.)

```
> This is a block quote. This
> paragraph has two lines.
>
> 1. This is a list inside a block quote.
> 2. Second item.
```

A "lazy" form, which requires the > character only on the first line of each block, is also allowed:

```
> This is a block quote. This
paragraph has two lines.

> 1. This is a list inside a block quote.
2. Second item.
```

Among the block elements that can be contained in a block quote are other block quotes. That is, block quotes can be nested:

```
> This is a block quote.
>
> > A block quote within a block quote.
```

If the > character is followed by an optional space, that space will be considered part of the block quote marker and not part of the indentation of the contents. Thus, to put an indented code block in a block quote, you need five spaces after the >:

```
>     code
```

Extension: `blank_before_blockquote`

Original Markdown syntax does not require a blank line before a block quote. Pandoc does require this (except, of course, at the beginning of the document). The reason for the requirement is that it is all too easy for a > to end up at the beginning of a line by accident (perhaps through line wrapping). So, unless the `markdown_strict` format is used, the following does not produce a nested block quote in pandoc:

```
> This is a block quote.
>> Not nested, since `blank_before_blockquote` is enabled by default
```

Verbatim (code) blocks

Indented code blocks

A block of text indented four spaces (or one tab) is treated as verbatim text: that is, special characters do not trigger special formatting, and all spaces and line breaks are preserved. For example,

```
    if (a > 3) {
      moveShip(5 * gravity, DOWN);
    }
```

The initial (four space or one tab) indentation is not considered part of the verbatim text, and is removed in the output.

Note: blank lines in the verbatim text need not begin with four spaces.

Fenced code blocks

Extension: `fenced_code_blocks`

In addition to standard indented code blocks, pandoc supports *fenced* code blocks. These begin with a row of three or more tildes (~) and end with a row of tildes that must be at least as long as the starting row. Everything between these lines is treated as code. No indentation is necessary:

```
~~~~~~~
if (a > 3) {
  moveShip(5 * gravity, DOWN);
}
~~~~~~~
```

Like regular code blocks, fenced code blocks must be separated from surrounding text by blank lines.

If the code itself contains a row of tildes or backticks, just use a longer row of tildes or backticks at the start and end:

```
~~~~~~~~~~~~~~~~
~~~~~~~~~~
code including tildes
~~~~~~~~~~
~~~~~~~~~~~~~~~~
```

Extension: `backtick_code_blocks`

Same as `fenced_code_blocks`, but uses backticks (`) instead of tildes (~).

Extension: `fenced_code_attributes`

Optionally, you may attach attributes to fenced or backtick code block using this syntax:

```
~~~~ {#mycode .haskell .numberLines startFrom="100"}
qsort []     = []
qsort (x:xs) = qsort (filter (< x) xs) ++ [x] ++
               qsort (filter (>= x) xs)
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

Here `mycode` is an identifier, `haskell` and `numberLines` are classes, and `startFrom` is an attribute with value `100`. Some output formats can use this information to do syntax highlighting. Currently, the only output formats that use this information are HTML, LaTeX, Docx, Ms, and PowerPoint. If highlighting is supported for your output format and language, then the code block above will appear highlighted, with numbered lines. (To see which languages are supported, type `pandoc --list-highlight-languages`.) Otherwise, the code block above will appear as follows:

```
<pre id="mycode" class="haskell numberLines" startFrom="100">
  <code>
  ...
  </code>
</pre>
```

The `numberLines` (or `number-lines`) class will cause the lines of the code block to be numbered, starting with `1` or the value of the `startFrom` attribute. The `lineAnchors` (or `line-anchors`) class will cause the lines to be clickable anchors in HTML output.

A shortcut form can also be used for specifying the language of the code block:

```
```haskell
qsort [] = []
```
```

This is equivalent to:

```
``` {.haskell}
qsort [] = []
```
```

This shortcut form may be combined with attributes:

```
```haskell {.numberLines}
qsort [] = []
```
```

Which is equivalent to:

```
``` {.haskell .numberLines}
qsort [] = []
```
```

If the `fenced_code_attributes` extension is disabled, but input contains class attribute(s) for the code block, the first class attribute will be printed after the opening fence as a bare word.

To prevent all highlighting, use the `--syntax-highlighting=none` option. To set the highlighting style or method, use `--syntax-highlighting`. For more information on highlighting, see Syntax highlighting, below.

Line blocks

Extension: `line_blocks`

A line block is a sequence of lines beginning with a vertical bar (`|`) followed by a space. The division into lines will be preserved in the output, as will any leading spaces; otherwise, the lines will be formatted as Markdown. This is useful for verse and addresses:

```
| The limerick packs laughs anatomical
| In space that is quite economical.
|    But the good ones I've seen
|    So seldom are clean
| And the clean ones so seldom are comical

| 200 Main St.
| Berkeley, CA 94718
```

The lines can be hard-wrapped if needed, but the continuation line must begin with a space.

```
| The Right Honorable Most Venerable and Righteous Samuel L.
  Constable, Jr.
| 200 Main St.
| Berkeley, CA 94718
```

Inline formatting (such as emphasis) is allowed in the content (though it can't cross line boundaries). Block-level formatting (such as block quotes or lists) is not recognized.

This syntax is borrowed from reStructuredText.

Lists

Bullet lists

A bullet list is a list of bulleted list items. A bulleted list item begins with a bullet (∗, +, or −). Here is a simple example:

```
∗ one
∗ two
∗ three
```

This will produce a "compact" list. If you want a "loose" list, in which each item is formatted as a paragraph, put spaces between the items:

```
∗ one

∗ two

∗ three
```

The bullets need not be flush with the left margin; they may be indented one, two, or three spaces. The bullet must be followed by whitespace.

List items look best if subsequent lines are flush with the first line (after the bullet):

```
∗ here is my first
  list item.
∗ and my second.
```

But Markdown also allows a "lazy" format:

```
∗ here is my first
list item.
∗ and my second.
```

Block content in list items

A list item may contain multiple paragraphs and other block-level content. However, subsequent paragraphs must be preceded by a blank line and indented to line up with the first non-space content after the list marker.

```
  * First paragraph.

    Continued.

  * Second paragraph. With a code block, which must be indented
    eight spaces:

        { code }
```

Exception: if the list marker is followed by an indented code block, which must begin 5 spaces after the list marker, then subsequent paragraphs must begin two columns after the last character of the list marker:

```
*     code

  continuation paragraph
```

List items may include other lists. In this case the preceding blank line is optional. The nested list must be indented to line up with the first non-space character after the list marker of the containing list item.

```
* fruits
  + apples
    - macintosh
    - red delicious
  + pears
  + peaches
* vegetables
  + broccoli
  + chard
```

As noted above, Markdown allows you to write list items "lazily," instead of indenting continuation lines. However, if there are multiple paragraphs or other blocks in a list item, the first line of each must be indented.

```
+ A lazy, lazy, list
item.

+ Another one; this looks
bad but is legal.

    Second paragraph of second
list item.
```

Ordered lists

Ordered lists work just like bulleted lists, except that the items begin with enumerators rather than bullets.

In original Markdown, enumerators are decimal numbers followed by a period and a space. The numbers themselves are ignored, so there is no difference between this list:

```
1.  one
2.  two
3.  three
```

and this one:

```
5.  one
7.  two
1.  three
```

Extension: `fancy_lists`

Unlike original Markdown, pandoc allows ordered list items to be marked with uppercase and lowercase letters and roman numerals, in addition to Arabic numerals. List markers may be enclosed in parentheses or followed by a single right-parenthesis or period. They must be separated from the text that follows by at least one space, and, if the list marker is a capital letter with a period, by at least two spaces.[1]

[1] The point of this rule is to ensure that normal paragraphs starting with people's initials, like

```
B. Russell won a Nobel Prize (but not for "On Denoting").
```

do not get treated as list items.
This rule will not prevent

```
(C) 2007 Joe Smith
```

from being interpreted as a list item. In this case, a backslash escape can be used:

```
(C\) 2007 Joe Smith
```

The `fancy_lists` extension also allows '`#`' to be used as an ordered list marker in place of a numeral:

```
#. one
#. two
```

Note: the '`#`' ordered list marker doesn't work with `commonmark`.

Extension: `startnum`

Pandoc also pays attention to the type of list marker used, and to the starting number, and both of these are preserved where possible in the output format. Thus, the following yields a list with numbers followed by a single parenthesis, starting with 9, and a sublist with lowercase roman numerals:

```
 9)  Ninth
10)  Tenth
11)  Eleventh
       i. subone
      ii. subtwo
     iii. subthree
```

Pandoc will start a new list each time a different type of list marker is used. So, the following will create three lists:

```
(2) Two
(5) Three
1.  Four
*   Five
```

If default list markers are desired, use `#.`:

```
#.  one
#.  two
#.  three
```

Extension: `task_lists`

Pandoc supports task lists, using the syntax of GitHub-Flavored Markdown.

```
- [ ] an unchecked task list item
- [x] checked item
```

Definition lists

Extension: `definition_lists`

Pandoc supports definition lists, using the syntax of PHP Markdown Extra with some extensions.[2]

```
Term 1

:   Definition 1

Term 2 with *inline markup*

:   Definition 2

        { some code, part of Definition 2 }

    Third paragraph of definition 2.
```

Each term must fit on one line, which may optionally be followed by a blank line, and must be followed by one or more definitions. A definition begins with a colon or tilde, which may be indented one or two spaces.

A term may have multiple definitions, and each definition may consist of one or more block elements (paragraph, code block, list, etc.), each indented four spaces or one tab stop. The body of the definition (not including the first line) should be indented four spaces. However, as with other Markdown lists, you can "lazily" omit indentation except at the beginning of a paragraph or other block element:

```
Term 1

:   Definition
with lazy continuation.

    Second paragraph of the definition.
```

[2] I have been influenced by the suggestions of David Wheeler.

If you leave space before the definition (as in the example above), the text of the definition will be treated as a paragraph. In some output formats, this will mean greater spacing between term/definition pairs. For a more compact definition list, omit the space before the definition:

```
Term 1
  ~ Definition 1

Term 2
  ~ Definition 2a
  ~ Definition 2b
```

Note that space between items in a definition list is required.

Numbered example lists

Extension: `example_lists`

The special list marker `@` can be used for sequentially numbered examples. The first list item with a `@` marker will be numbered '1', the next '2', and so on, throughout the document. The numbered examples need not occur in a single list; each new list using `@` will take up where the last stopped. So, for example:

```
(@)  My first example will be numbered (1).
(@)  My second example will be numbered (2).

Explanation of examples.

(@)  My third example will be numbered (3).
```

Numbered examples can be labeled and referred to elsewhere in the document:

```
(@good)  This is a good example.

As (@good) illustrates, ...
```

The label can be any string of alphanumeric characters, underscores, or hyphens.

Continuation paragraphs in example lists must always be indented four spaces, regardless of the length of the list marker. That is, example lists always behave as if the `four_space_rule` extension is set. This is because example labels tend to be long, and indenting content to the first non-space character after the label would be awkward.

You can repeat an earlier numbered example by re-using its label:

```
(@foo) Sample sentence.

Intervening text...

This theory can explain the case we saw earlier (repeated):

(@foo) Sample sentence.
```

This only works reliably, though, if the repeated item is in a list by itself, because each numbered example list will be numbered continuously from its starting number.

Ending a list

What if you want to put an indented code block after a list?

```
-   item one
-   item two

    { my code block }
```

Trouble! Here pandoc (like other Markdown implementations) will treat `{ my code block }` as the second paragraph of item two, and not as a code block.

To "cut off" the list after item two, you can insert some non-indented content, like an HTML comment, which won't produce visible output in any format:

```
-   item one
-   item two

<!-- end of list -->

    { my code block }
```

You can use the same trick if you want two consecutive lists instead of one big list:

```
1.  one
2.  two
3.  three

<!-- -->

1.  uno
2.  dos
3.  tres
```

Horizontal rules

A line containing a row of three or more *, –, or _ characters (optionally separated by spaces) produces a horizontal rule:

```
* * * *

---------------
```

We strongly recommend that horizontal rules be separated from surrounding text by blank lines. If a horizontal rule is not followed by a blank line, pandoc may try to interpret the lines that follow as a YAML metadata block or a table.

Tables

Four kinds of tables may be used. The first three kinds presuppose the use of a fixed-width font, such as Courier. The fourth kind can be used with proportionally spaced fonts, as it does not require lining up columns.

Extension: `table_captions`

A caption may optionally be provided with all 4 kinds of tables (as illustrated in the examples below). A caption is a paragraph beginning with the string `Table:` (or `table:` or just `:`), which will be stripped off. It may appear either before or after the table.

Extension: `simple_tables`

Simple tables look like this:

```
  Right     Left     Center     Default
-------     ------ ----------   -------
     12     12        12            12
    123     123       123          123
      1     1          1             1

Table:  Demonstration of simple table syntax.
```

The header and table rows must each fit on one line. Column alignments are determined by the position of the header text relative to the dashed line below it:[3]

[3]This scheme is due to Michel Fortin, who proposed it on the Markdown discussion list.

- If the dashed line is flush with the header text on the right side but extends beyond it on the left, the column is right-aligned.
- If the dashed line is flush with the header text on the left side but extends beyond it on the right, the column is left-aligned.
- If the dashed line extends beyond the header text on both sides, the column is centered.
- If the dashed line is flush with the header text on both sides, the default alignment is used (in most cases, this will be left).

The table must end with a blank line, or a line of dashes followed by a blank line.

The column header row may be omitted, provided a dashed line is used to end the table. For example:

```
-------     ------ ----------   -------
     12     12        12             12
    123     123       123           123
      1     1          1              1
-------     ------ ----------   -------
```

When the header row is omitted, column alignments are determined on the basis of the first line of the table body. So, in the tables above, the columns would be right, left, center, and right aligned, respectively.

Extension: `multiline_tables`

Multiline tables allow header and table rows to span multiple lines of text (but cells that span multiple columns or rows of the table are not supported). Here is an example:

```
-------------------------------------------------------------
 Centered   Default           Right Left
  Header    Aligned         Aligned Aligned
----------- ------- --------------- -------------------------
   First    row                12.0 Example of a row that
                                    spans multiple lines.

  Second    row                 5.0 Here's another one. Note
                                    the blank line between
                                    rows.
-------------------------------------------------------------

Table: Here's the caption. It, too, may span
multiple lines.
```

These work like simple tables, but with the following differences:

- They must begin with a row of dashes, before the header text (unless the header row is omitted).
- They must end with a row of dashes, then a blank line.
- The rows must be separated by blank lines.

In multiline tables, the table parser pays attention to the widths of the columns, and the writers try to reproduce these relative widths in the output. So, if you find that one of the columns is too narrow in the output, try widening it in the Markdown source.

The header may be omitted in multiline tables as well as simple tables:

```
----------- ------- --------------- -------------------------
   First    row                12.0 Example of a row that
                                    spans multiple lines.

  Second    row                 5.0 Here's another one. Note
                                    the blank line between
                                    rows.
----------- ------- --------------- -------------------------

: Here's a multiline table without a header.
```

It is possible for a multiline table to have just one row, but the row should be followed by a blank line (and then the row of dashes that ends the table), or the table may be interpreted as a simple table.

Extension: `grid_tables`

Grid tables look like this:

```
: Sample grid table.

+---------------+---------------+--------------------+
| Fruit         | Price         | Advantages         |
+===============+===============+====================+
| Bananas       | $1.34         | - built-in wrapper |
|               |               | - bright color     |
+---------------+---------------+--------------------+
| Oranges       | $2.10         | - cures scurvy     |
|               |               | - tasty            |
+---------------+---------------+--------------------+
```

The row of =s separates the header from the table body, and can be omitted for a headerless table. The cells of grid tables may contain arbitrary block elements (multiple paragraphs, code blocks, lists, etc.).

Cells can span multiple columns or rows:

```
+--------------------+----------+
| Property           | Earth    |
+============+=======+==========+
|            | min   | -89.2 °C |
| Temperature +-------+----------+
| 1961-1990  | mean  | 14 °C    |
|            +-------+----------+
|            | max   | 56.7 °C  |
+------------+-------+----------+
```

A table header may contain more than one row:

```
+--------------------+-----------------------+
| Location           | Temperature 1961-1990 |
|                    | in degree Celsius     |
|                    +-------+-------+-------+
|                    | min   | mean  | max   |
+====================+=======+=======+=======+
| Antarctica         | -89.2 | N/A   | 19.8  |
+--------------------+-------+-------+-------+
| Earth              | -89.2 | 14    | 56.7  |
+--------------------+-------+-------+-------+
```

Alignments can be specified as with pipe tables, by putting colons at the boundaries of the separator line after the header:

```
+--------------+--------------+--------------------+
| Right        | Left         | Centered           |
+=============:+:=============+:==================:+
| Bananas      | $1.34        | built-in wrapper   |
+--------------+--------------+--------------------+
```

For headerless tables, the colons go on the top line instead:

```
+-------------:+:-------------+:------------------:+
| Right        | Left         | Centered           |
+--------------+--------------+--------------------+
```

A table foot can be defined by enclosing it with separator lines that use = instead of –:

```
+---------------+---------------+
| Fruit         | Price         |
+===============+===============+
| Bananas       | $1.34         |
+---------------+---------------+
| Oranges       | $2.10         |
+===============+===============+
| Sum           | $3.44         |
+===============+===============+
```

The foot must always be placed at the very bottom of the table.

Grid tables can be created easily using Emacs' table-mode (`M-x table-insert`).

Extension: `pipe_tables`

Pipe tables look like this:

```
| Right | Left | Default | Center |
|------:|:-----|---------|:------:|
|   12  |  12  |    12   |    12  |
|  123  |  123 |   123   |   123  |
|    1  |    1 |     1   |     1  |

  : Demonstration of pipe table syntax.
```

The syntax is identical to PHP Markdown Extra tables. The beginning and ending pipe characters are optional, but pipes are required between all columns. The colons indicate column alignment as shown. The header cannot be omitted. To simulate a headerless table, include a header with blank cells.

Since the pipes indicate column boundaries, columns need not be vertically aligned, as they are in the above example. So, this is a perfectly legal (though ugly) pipe table:

```
fruit| price
-----|-----:
apple|2.05
pear|1.37
orange|3.09
```

The cells of pipe tables cannot contain block elements like paragraphs and lists, and cannot span multiple lines. If any line of the Markdown source is longer than the column width (see `--columns`), then the table will take up the full text width and the cell contents will wrap, with the relative cell widths determined by the number of dashes in the line separating the table header from the table body. (For example `---|-` would make the first column 3/4 and the second column 1/4 of the full text width.) On the other hand, if no lines are wider than column width, then cell contents will not be wrapped, and the cells will be sized to their contents.

Note: pandoc also recognizes pipe tables of the following form, as can be produced by Emacs' orgtbl-mode:

```
| One | Two   |
|-----+-------|
| my  | table |
| is  | nice  |
```

The difference is that `+` is used instead of `|`. Other orgtbl features are not supported. In particular, to get non-default column alignment, you'll need to add colons as above.

Extension: `table_attributes`

Attributes may be attached to tables by including them at the end of the caption. (For the syntax, see `header_attributes`.)

```
  : Here's the caption. {#ident .class key="value"}
```

Metadata blocks

Extension: `pandoc_title_block`

If the file begins with a title block

```
% title
% author(s) (separated by semicolons)
% date
```

it will be parsed as bibliographic information, not regular text. (It will be used, for example, in the title of standalone LaTeX or HTML output.) The block may contain just a title, a date and an author, or all three elements. If you want to include an author but no title, or a title and a date but no author, you need a blank line:

```
%
% Author

% My title
%
% June 15, 2006
```

The title may occupy multiple lines, but continuation lines must begin with leading space, thus:

```
% My title
  on multiple lines
```

If a document has multiple authors, the authors may be put on separate lines with leading space, or separated by semicolons, or both. So, all of the following are equivalent:

```
% Author One
  Author Two

% Author One; Author Two

% Author One;
  Author Two
```

The date must fit on one line.

All three metadata fields may contain standard inline formatting (italics, links, footnotes, etc.).

Title blocks will always be parsed, but they will affect the output only when the `--standalone` (`-s`) option is chosen. In HTML output, titles will appear twice: once in the document head—this is the title that will appear at the top of the window in a browser—and once at the beginning of the document body. The title in the document head can have an optional prefix attached (`--title-prefix` or `-T` option). The title in the body appears as an H1 element with class "title", so it can be suppressed or reformatted with CSS. If a title prefix is specified with `-T` and no title block appears in the document, the title prefix will be used by itself as the HTML title.

The man page writer extracts a title, man page section number, and other header and footer information from the title line. The title is assumed to be the first word on the title line, which may optionally end with a (single-digit) section number in parentheses. (There should be no space between the title and the parentheses.) Anything after this is assumed to be additional footer and header text. A single pipe character (`|`) should be used to separate the footer text from the header text. Thus,

```
% PANDOC(1)
```

will yield a man page with the title `PANDOC` and section 1.

```
% PANDOC(1) Pandoc User Manuals
```

will also have "Pandoc User Manuals" in the footer.

```
% PANDOC(1) Pandoc User Manuals | Version 4.0
```

will also have "Version 4.0" in the header.

Extension: `yaml_metadata_block`

A YAML metadata block is a valid YAML object, delimited by a line of three hyphens (`---`) at the top and a line of three hyphens (`---`) or three dots (`...`) at the bottom. The initial line `---` must not be followed by a blank line. A YAML metadata block may occur anywhere in the document, but if it is not at the beginning, it must be preceded by a blank line.

Note that, because of the way pandoc concatenates input files when several are provided, you may also keep the metadata in a separate YAML file and pass it to pandoc as an argument, along with your Markdown files:

```
pandoc chap1.md chap2.md chap3.md metadata.yaml -s -o book.html
```

Just be sure that the YAML file begins with `---` and ends with `---` or `...`. Alternatively, you can use the `--metadata-file` option. Using that approach however, you cannot reference content (like footnotes) from the main Markdown input document.

Metadata will be taken from the fields of the YAML object and added to any existing document metadata. Metadata can contain lists and objects (nested arbitrarily), but all string scalars will be interpreted as Markdown. Fields with names ending in an underscore will be ignored by pandoc. (They may be given a role by external processors.) Field names must not be interpretable as YAML numbers or boolean values (so, for example, `yes`, `True`, and `15` cannot be used as field names).

A document may contain multiple metadata blocks. If two metadata blocks attempt to set the same field, the value from the second block will be taken.

Each metadata block is handled internally as an independent YAML document. This means, for example, that any YAML anchors defined in a block cannot be referenced in another block.

When pandoc is used with `-t markdown` to create a Markdown document, a YAML metadata block will be produced only if the `-s/--standalone` option is used. All of the metadata will appear in a single block at the beginning of the document.

Note that YAML escaping rules must be followed. Thus, for example, if a title contains a colon, it must be quoted, and if it contains a backslash escape, then it must be ensured that it is not treated as a YAML escape sequence. The pipe character (`|`) can be used to begin an indented block that will be interpreted literally, without need for escaping. This form is necessary when the field contains blank lines or block-level formatting:

```
---
title:  'This is the title: it contains a colon'
author:
- Author One
- Author Two
keywords: [nothing, nothingness]
abstract: |
  This is the abstract.

  It consists of two paragraphs.
...
```

The literal block after the `|` must be indented relative to the line containing the `|`. If it is not, the YAML will be invalid and pandoc will not interpret it as metadata. For an overview of the complex rules governing YAML, see the Wikipedia entry on YAML syntax.

Template variables will be set automatically from the metadata. Thus, for example, in writing HTML, the variable `abstract` will be set to the HTML equivalent of the Markdown in the `abstract` field:

```
<p>This is the abstract.</p>
<p>It consists of two paragraphs.</p>
```

Variables can contain arbitrary YAML structures, but the template must match this structure. The `author` variable in the default templates expects a simple list or string, but can be changed to support more complicated structures. The following combination, for example, would add an affiliation to the author if one is given:

```
---
title: The document title
author:
- name: Author One
  affiliation: University of Somewhere
```

```
- name: Author Two
  affiliation: University of Nowhere
...
```

To use the structured authors in the example above, you would need a custom template:

```
$for(author)$
$if(author.name)$
$author.name$$if(author.affiliation)$ ($author.affiliation$)$endif$
$else$
$author$
$endif$
$endfor$
```

Raw content to include in the document's header may be specified using `header-includes`; however, it is important to mark up this content as raw code for a particular output format, using the `raw_attribute` extension, or it will be interpreted as Markdown. For example:

```
header-includes:
- |
  ```{=latex}
  \let\oldsection\section
  \renewcommand{\section}[1]{\clearpage\oldsection{#1}}
  ```
```

Note: the `yaml_metadata_block` extension works with `commonmark` as well as `markdown` (and it is enabled by default in `gfm` and `commonmark_x`). However, in these formats the following restrictions apply:

- The YAML metadata block must occur at the beginning of the document (and there can be only one). If multiple files are given as arguments to pandoc, only the first can be a YAML metadata block.
- The leaf nodes of the YAML structure are parsed in isolation from each other and from the rest of the document. So, for example, you can't use a reference link in these contexts if the link definition is somewhere else in the document.

Backslash escapes

Extension: `all_symbols_escapable`

Except inside a code block or inline code, any punctuation or space character preceded by a backslash will be treated literally, even if it would normally indicate formatting. Thus, for example, if one writes

```
*\*hello\**
```

one will get

```
<em>*hello*</em>
```

instead of

```
<strong>hello</strong>
```

This rule is easier to remember than original Markdown's rule, which allows only the following characters to be backslash-escaped:

```
\`*_{}[]()>#+-.!
```

(However, if the `markdown_strict` format is used, the original Markdown rule will be used.)

A backslash-escaped space is parsed as a nonbreaking space. In TeX output, it will appear as ~. In HTML and XML output, it will appear as a literal unicode nonbreaking space character (note that it will thus actually look "invisible" in the generated HTML source; you can still use the `--ascii` command-line option to make it appear as an explicit entity).

A backslash-escaped newline (i.e. a backslash occurring at the end of a line) is parsed as a hard line break. It will appear in TeX output as `\\` and in HTML as `<br />`. This is a nice alternative to Markdown's "invisible" way of indicating hard line breaks using two trailing spaces on a line.

Backslash escapes do not work in verbatim contexts.

Inline formatting

Emphasis

To *emphasize* some text, surround it with * or _, like this:

```
This text is _emphasized with underscores_, and this
is *emphasized with asterisks*.
```

Double * or _ produces **strong emphasis**:

```
This is **strong emphasis** and __with underscores__.
```

A ∗ or _ character surrounded by spaces, or backslash-escaped, will not trigger emphasis:

```
This is ∗ not emphasized ∗, and \∗neither is this\∗.
```

Extension: `intraword_underscores`

Because _ is sometimes used inside words and identifiers, pandoc does not interpret a _ surrounded by alphanumeric characters as an emphasis marker. If you want to emphasize just part of a word, use ∗:

```
feas∗ible∗, not feas∗able∗.
```

Strikeout

Extension: `strikeout`

To strike out a section of text with a horizontal line, begin and end it with ~~. Thus, for example,

```
This ~~is deleted text.~~
```

Superscripts and subscripts

Extension: `superscript`, `subscript`

Superscripts may be written by surrounding the superscripted text by ^ characters; subscripts may be written by surrounding the subscripted text by ~ characters. Thus, for example,

```
H~2~O is a liquid.  2^10^ is 1024.
```

The text between `^...^` or `~...~` may not contain spaces or newlines. If the superscripted or subscripted text contains spaces, these spaces must be escaped with backslashes. (This is to prevent accidental superscripting and subscripting through the ordinary use of ~ and ^, and also bad interactions with footnotes.) Thus, if you want the letter P with 'a cat' in subscripts, use `P~a\ cat~`, not `P~a cat~`.

Verbatim

To make a short span of text verbatim, put it inside backticks:

```
What is the difference between `>>=` and `>>`?
```

If the verbatim text includes a backtick, use double backticks:

```
Here is a literal backtick `` ` ``.
```

(The spaces after the opening backticks and before the closing backticks will be ignored.)

The general rule is that a verbatim span starts with a string of consecutive backticks (optionally followed by a space) and ends with a string of the same number of backticks (optionally preceded by a space).

Note that backslash-escapes (and other Markdown constructs) do not work in verbatim contexts:

```
This is a backslash followed by an asterisk: `\*`.
```

Extension: `inline_code_attributes`

Attributes can be attached to verbatim text, just as with fenced code blocks:

```
`<$>`{.haskell}
```

Underline

To underline text, use the `underline` class:

```
[Underline]{.underline}
```

Or, without the `bracketed_spans` extension (but with `native_spans`):

```
<span class="underline">Underline</span>
```

This will work in all output formats that support underline.

Small caps

To write small caps, use the `smallcaps` class:

```
[Small caps]{.smallcaps}
```

Or, without the `bracketed_spans` extension:

```
<span class="smallcaps">Small caps</span>
```

For compatibility with other Markdown flavors, CSS is also supported:

```
<span style="font-variant:small-caps;">Small caps</span>
```

This will work in all output formats that support small caps.

Highlighting

To highlight text, use the `mark` class:

```
[Mark]{.mark}
```

Or, without the `bracketed_spans` extension (but with `native_spans`):

```
<span class="mark">Mark</span>
```

This will work in all output formats that support highlighting.

Math

Extension: `tex_math_dollars`

Anything between two `$` characters will be treated as TeX math. The opening `$` must have a non-space character immediately to its right, while the closing `$` must have a non-space character immediately to its left, and must not be followed immediately by a digit. Thus, `$20,000 and $30,000` won't parse as math. If for some reason you need to enclose text in literal `$` characters, backslash-escape them and they won't be treated as math delimiters.

For display math, use `$$` delimiters. (In this case, the delimiters may be separated from the formula by whitespace. However, there can be no blank lines between the opening and closing `$$` delimiters.)

TeX math will be printed in all output formats. How it is rendered depends on the output format:

LaTeX It will appear verbatim surrounded by `\(...\)` (for inline math) or `\[...\]` (for display math).

Markdown, Emacs Org mode, ConTeXt, ZimWiki It will appear verbatim surrounded by `$...$` (for inline math) or `$$...$$` (for display math).

XWiki It will appear verbatim surrounded by `{{formula}}..{{/formula}}`.

reStructuredText It will be rendered using an interpreted text role `:math:`.

AsciiDoc For AsciiDoc output math will appear verbatim surrounded by `latexmath:[...]`. For `asciidoc_legacy` the bracketed material will also include inline or display math delimiters.

Texinfo It will be rendered inside a `@math` command.

roff man, Jira markup It will be rendered verbatim without `$`'s.

MediaWiki, DokuWiki It will be rendered inside `<math>` tags.

Textile It will be rendered inside `<span class="math">` tags.

RTF, OpenDocument It will be rendered, if possible, using Unicode characters, and will otherwise appear verbatim.

ODT It will be rendered, if possible, using MathML.

DocBook If the `--mathml` flag is used, it will be rendered using MathML in an `inlineequation` or `informalequation` tag. Otherwise it will be rendered, if possible, using Unicode characters.

Docx and PowerPoint It will be rendered using OMML math markup.

FictionBook2 If the `--webtex` option is used, formulas are rendered as images using CodeCogs or other compatible web service, downloaded and embedded in the e-book. Otherwise, they will appear verbatim.

HTML, Slidy, DZSlides, S5, EPUB The way math is rendered in HTML will depend on the command-line options selected. Therefore see Math rendering in HTML above.

Raw HTML

Extension: `raw_html`

Markdown allows you to insert raw HTML (or DocBook) anywhere in a document (except verbatim contexts, where `<`, `>`, and `&` are interpreted literally). (Technically this is not an extension, since standard Markdown allows it, but it has been made an extension so that it can be disabled if desired.)

The raw HTML is passed through unchanged in HTML, S5, Slidy, Slideous, DZSlides, EPUB, Markdown, CommonMark, Emacs Org mode, and Textile output, and suppressed in other formats.

For a more explicit way of including raw HTML in a Markdown document, see the `raw_attribute` extension.

In the CommonMark format, if `raw_html` is enabled, superscripts, subscripts, strikeouts and small capitals will be represented as HTML. Otherwise, plain-text fallbacks will be used. Note that even if `raw_html` is disabled, tables will be rendered with HTML syntax if they cannot use pipe syntax.

Extension: `markdown_in_html_blocks`

Original Markdown allows you to include HTML "blocks": blocks of HTML between balanced tags that are separated from the surrounding text with blank lines, and start and end at the left margin. Within these blocks, everything is interpreted as HTML, not Markdown; so (for example), `*` does not signify emphasis.

Pandoc behaves this way when the `markdown_strict` format is used; but by default, pandoc interprets material between HTML block tags as Markdown. Thus, for example, pandoc will turn

```
<table>
<tr>
<td>*one*</td>
<td>[a link](https://google.com)</td>
</tr>
</table>
```

into

```
<table>
<tr>
<td><em>one</em></td>
<td><a href="https://google.com">a link</a></td>
</tr>
</table>
```

whereas `Markdown.pl` will preserve it as is.

There is one exception to this rule: text between `<script>`, `<style>`, `<pre>`, and `<textarea>` tags is not interpreted as Markdown.

This departure from original Markdown should make it easier to mix Markdown with HTML block elements. For example, one can surround a block of Markdown text with `<div>` tags without preventing it from being interpreted as Markdown.

Extension: `native_divs`

Use native pandoc `Div` blocks for content inside `<div>` tags. For the most part this should give the same output as `markdown_in_html_blocks`, but it makes it easier to write pandoc filters to manipulate groups of blocks.

Extension: `native_spans`

Use native pandoc `Span` blocks for content inside `<span>` tags. For the most part this should give the same output as `raw_html`, but it makes it easier to write pandoc filters to manipulate groups of inlines.

Extension: `raw_tex`

In addition to raw HTML, pandoc allows raw LaTeX, TeX, and ConTeXt to be included in a document. Inline TeX commands will be preserved and passed unchanged to the LaTeX and ConTeXt writers. Thus, for example, you can use LaTeX to include BibTeX citations:

```
This result was proved in \cite{jones.1967}.
```

Note that in LaTeX environments, like

```
\begin{tabular}{|l|l|}\hline
Age & Frequency \\ \hline
18--25  & 15 \\
26--35  & 33 \\
36--45  & 22 \\ \hline
\end{tabular}
```

the material between the begin and end tags will be interpreted as raw LaTeX, not as Markdown.

For a more explicit and flexible way of including raw TeX in a Markdown document, see the `raw_attribute` extension.

Inline LaTeX is ignored in output formats other than Markdown, LaTeX, Emacs Org mode, and ConTeXt.

Generic raw attribute

Extension: `raw_attribute`

Inline spans and fenced code blocks with a special kind of attribute will be parsed as raw content with the designated format. For example, the following produces a raw roff `ms` block:

```
```{=ms}
.MYMACRO
blah blah
```
```

And the following produces a raw `html` inline element:

```
This is `<a>html</a>`{=html}
```

This can be useful to insert raw xml into `docx` documents, e.g. a pagebreak:

```
```{=openxml}
<w:p>
  <w:r>
    <w:br w:type="page"/>
  </w:r>
</w:p>
```
```

The format name should match the target format name (see `-t`/`--to`, above, for a list, or use `pandoc --list-output-formats`). Use `openxml` for `docx` output, `opendocument` for `odt` output, `html5` for `epub3` output, `html4` for `epub2` output, and `latex`, `beamer`, `ms`, or `html5` for `pdf` output (depending on what you use for `--pdf-engine`).

This extension presupposes that the relevant kind of inline code or fenced code block is enabled. Thus, for example, to use a raw attribute with a backtick code block, `backtick_code_blocks` must be enabled.

The raw attribute cannot be combined with regular attributes.

LaTeX macros

Extension: `latex_macros`

When this extension is enabled, pandoc will parse LaTeX macro definitions and apply the resulting macros to all LaTeX math and raw LaTeX. So, for example, the following will work in all output formats, not just LaTeX:

```
\newcommand{\tuple}[1]{\langle #1 \rangle}

$\tuple{a, b, c}$
```

Note that LaTeX macros will not be applied if they occur inside a raw span or block marked with the `raw_attribute` extension.

When `latex_macros` is disabled, the raw LaTeX and math will not have macros applied. This is usually a better approach when you are targeting LaTeX or PDF.

Macro definitions in LaTeX will be passed through as raw LaTeX only if `latex_macros` is not enabled. Macro definitions in Markdown source (or other formats allowing `raw_tex`) will be passed through regardless of whether `latex_macros` is enabled.

Links

Markdown allows links to be specified in several ways.

Automatic links

If you enclose a URL or email address in pointy brackets, it will become a link:

```
<https://google.com>
<sam@green.eggs.ham>
```

Inline links

An inline link consists of the link text in square brackets, followed by the URL in parentheses. (Optionally, the URL can be followed by a link title, in quotes.)

```
This is an [inline link](/url), and here's [one with
a title](https://fsf.org "click here for a good time!").
```

There can be no space between the bracketed part and the parenthesized part. The link text can contain formatting (such as emphasis), but the title cannot.

Email addresses in inline links are not autodetected, so they have to be prefixed with `mailto`:

```
[Write me!](mailto:sam@green.eggs.ham)
```

Reference links

An *explicit* reference link has two parts, the link itself and the link definition, which may occur elsewhere in the document (either before or after the link).

The link consists of link text in square brackets, followed by a label in square brackets. (There cannot be space between the two unless the `spaced_reference_links` extension is enabled.) The link definition consists of the bracketed label, followed by a colon and a space, followed by the URL, and optionally (after a space) a link title either in quotes or in parentheses. The label must not be parseable as a citation (assuming the `citations` extension is enabled): citations take precedence over link labels.

Here are some examples:

```
[my label 1]: /foo/bar.html  "My title, optional"
[my label 2]: /foo
[my label 3]: https://fsf.org (The Free Software Foundation)
[my label 4]: /bar#special  'A title in single quotes'
```

The URL may optionally be surrounded by angle brackets:

```
[my label 5]: <http://foo.bar.baz>
```

The title may go on the next line:

```
[my label 3]: https://fsf.org
  "The Free Software Foundation"
```

Note that link labels are not case sensitive. So, this will work:

```
Here is [my link][FOO]

[Foo]: /bar/baz
```

In an *implicit* reference link, the second pair of brackets is empty:

```
See [my website][].

[my website]: http://foo.bar.baz
```

Note: In `Markdown.pl` and most other Markdown implementations, reference link definitions cannot occur in nested constructions such as list items or block quotes. Pandoc lifts this arbitrary-seeming restriction. So the following is fine in pandoc, though not in most other implementations:

```
> My block [quote].
>
> [quote]: /foo
```

Extension: `shortcut_reference_links`

In a *shortcut* reference link, the second pair of brackets may be omitted entirely:

```
See [my website].

[my website]: http://foo.bar.baz
```

Internal links

To link to another section of the same document, use the automatically generated identifier (see Heading identifiers). For example:

```
See the [Introduction](#introduction).
```

or

```
See the [Introduction].

[Introduction]: #introduction
```

Internal links are currently supported for HTML formats (including HTML slide shows and EPUB), LaTeX, and ConTeXt.

Images

A link immediately preceded by a `!` will be treated as an image. The link text will be used as the image's alt text:

```
![la lune](lalune.jpg "Voyage to the moon")

![movie reel]

[movie reel]: movie.gif
```

Extension: `implicit_figures`

An image with nonempty alt text, occurring by itself in a paragraph, will be rendered as a figure with a caption. The image's alt text will be used as the caption.

```
![This is the caption](/url/of/image.png)
```

How this is rendered depends on the output format. Some output formats (e.g. RTF) do not yet support figures. In those formats, you'll just get an image in a paragraph by itself, with no caption. For LaTeX output, you can specify a figure's positioning by adding the `latex-placement` attribute.

```
![This is the caption](/url/of/image.png){latex-placement="ht"}
```

If you just want a regular inline image, just make sure it is not the only thing in the paragraph. One way to do this is to insert a nonbreaking space after the image:

```
![This image won't be a figure](/url/of/image.png)\
```

Note that in reveal.js slide shows, an image in a paragraph by itself that has the `r-stretch` class will fill the screen, and the caption and figure tags will be omitted.

Extension: `link_attributes`

Attributes can be set on links and images:

```
An inline ![image](foo.jpg){#id .class width=30 height=20px}
and a reference ![image][ref] with attributes.

[ref]: foo.jpg "optional title" {#id .class key=val key2="val 2"}
```

(This syntax is compatible with PHP Markdown Extra when only `#id` and `.class` are used.)

For HTML and EPUB, all known HTML5 attributes except `width` and `height` (but including `srcset` and `sizes`) are passed through as is. Unknown attributes are passed through as custom attributes, with `data-` prepended. The other writers ignore attributes that are not specifically supported by their output format.

The `width` and `height` attributes on images are treated specially. When used without a unit, the unit is assumed to be pixels. However, any of the following unit identifiers can be used: `px`, `cm`, `mm`, `in`, `inch` and `%`. There must not be any spaces between the number and the unit. For example:

```
![](file.jpg){ width=50% }
```

- Dimensions may be converted to a form that is compatible with the output format (for example, dimensions given in pixels will be converted to inches when converting HTML to LaTeX). Conversion between pixels and physical measurements is affected by the `--dpi` option (by default, 96 dpi is assumed, unless the image itself contains dpi information).
- The `%` unit is generally relative to some available space. For example the above example will render to the following.
  - HTML: `<img href="file.jpg" style="width: 50%;" />`
  - LaTeX: `\includegraphics[width=0.5\textwidth,height=\textheight]{file.jpg}` (If you're using a custom template, you need to configure `graphicx` as in the default template.)
  - ConTeXt: `\externalfigure[file.jpg][width=0.5\textwidth]`
- Some output formats have a notion of a class (ConTeXt) or a unique identifier (LaTeX `\caption`), or both (HTML).
- When no `width` or `height` attributes are specified, the fallback is to look at the image resolution and the dpi metadata embedded in the image file.

Divs and Spans

Using the `native_divs` and `native_spans` extensions (see above), HTML syntax can be used as part of Markdown to create native `Div` and `Span` elements in the pandoc AST (as opposed to raw HTML). However, there is also nicer syntax available:

Extension: `fenced_divs`

Allow special fenced syntax for native `Div` blocks. A Div starts with a fence containing at least three consecutive colons plus some attributes. The attributes may optionally be followed by another string of consecutive colons.

Note: the `commonmark` parser doesn't permit colons after the attributes.

The attribute syntax is exactly as in fenced code blocks (see Extension: `fenced_code_attributes`). As with fenced code blocks, one can use either attributes in curly braces or a single unbraced word, which will be treated as a class name. The Div ends with another line containing a string of at least three consecutive colons. The fenced Div should be separated by blank lines from preceding and following blocks.

Example:

```
::::: {#special .sidebar}
Here is a paragraph.

And another.
:::::
```

Fenced divs can be nested. Opening fences are distinguished because they *must* have attributes:

```
::: Warning ::::::
This is a warning.

::: Danger
This is a warning within a warning.
:::
::::::::::::::::
```

Fences without attributes are always closing fences. Unlike with fenced code blocks, the number of colons in the closing fence need not match the number in the opening fence. However, it can be helpful for visual clarity to use fences of different lengths to distinguish nested divs from their parents.

Extension: `bracketed_spans`

A bracketed sequence of inlines, as one would use to begin a link, will be treated as a `Span` with attributes if it is followed immediately by attributes:

```
[This is *some text*]{.class key="val"}
```

Footnotes

Extension: `footnotes`

Pandoc's Markdown allows footnotes, using the following syntax:

```
Here is a footnote reference,[^1] and another.[^longnote]

[^1]: Here is the footnote.

[^longnote]: Here's one with multiple blocks.

    Subsequent paragraphs are indented to show that they
belong to the previous footnote.

        { some.code }

    The whole paragraph can be indented, or just the first
    line.  In this way, multi-paragraph footnotes work like
    multi-paragraph list items.

This paragraph won't be part of the note, because it
isn't indented.
```

The identifiers in footnote references may not contain spaces, tabs, newlines, or the characters `^`, `[`, or `]`. These identifiers are used only to correlate the footnote reference with the note itself; in the output, footnotes will be numbered sequentially.

The footnotes themselves need not be placed at the end of the document. They may appear anywhere except inside other block elements (lists, block quotes, tables, etc.). Each footnote should be separated from surrounding content (including other footnotes) by blank lines.

Extension: `inline_notes`

Inline footnotes are also allowed (though, unlike regular notes, they cannot contain multiple paragraphs). The syntax is as follows:

```
Here is an inline note.^[Inline notes are easier to write, since
you don't have to pick an identifier and move down to type the
note.]
```

Inline and regular footnotes may be mixed freely.

Citation syntax

Extension: `citations`

To cite a bibliographic item with an identifier foo, use the syntax `@foo`. Normal citations should be included in square brackets, with semicolons separating distinct items:

```
Blah blah [@doe99; @smith2000; @smith2004].
```

How this is rendered depends on the citation style. In an author-date style, it might render as

```
Blah blah (Doe 1999, Smith 2000, 2004).
```

In a footnote style, it might render as

```
Blah blah.[^1]

[^1]:  John Doe, "Frogs," *Journal of Amphibians* 44 (1999);
Susan Smith, "Flies," *Journal of Insects* (2000);
Susan Smith, "Bees," *Journal of Insects* (2004).
```

See the CSL user documentation for more information about CSL styles and how they affect rendering.

Unless a citation key starts with a letter, digit, or _, and contains only alphanumerics and single internal punctuation characters (`:.#$%&-+?<>~/`), it must be surrounded by curly braces, which are not considered part of the key. In `@Foo_bar.baz.`, the key is `Foo_bar.baz` because the final period is not *internal* punctuation, so it is not included in the key. In `@{Foo_bar.baz.}`, the key is `Foo_bar.baz.`, including the final period.

In `@Foo_bar--baz`, the key is `Foo_bar` because the repeated internal punctuation characters terminate the key. The curly braces are recommended if you use URLs as keys: `[@{https://example.com/bib?name=foobar&date=2000}, p. 33]`.

Citation items may optionally include a prefix, a locator, and a suffix. In

```
Blah blah [see @doe99, pp. 33–35 and *passim*; @smith04, chap. 1].
```

the first item (`doe99`) has prefix `see`, locator `pp. 33–35`, and suffix `and *passim*`. The second item (`smith04`) has locator `chap. 1` and no prefix or suffix.

Pandoc uses some heuristics to separate the locator from the rest of the subject. It is sensitive to the locator terms defined in the CSL locale files. Either abbreviated or unabbreviated forms are accepted. In the `en-US` locale, locator terms can be written in either singular or plural forms, as `book`, `bk.`/`bks.`; `chapter`, `chap.`/`chaps.`; `column`, `col.`/`cols.`; `figure`, `fig.`/`figs.`; `folio`, `fol.`/`fols.`; `number`, `no.`/`nos.`; `line`, `l.`/`ll.`; `note`, `n.`/`nn.`; `opus`, `op.`/`opp.`; `page`, `p.`/`pp.`; `paragraph`, `para.`/`paras.`; `part`, `pt.`/`pts.`; `section`, `sec.`/`secs.`; `sub verbo`, `s.v.`/`s.vv.`; `verse`, `v.`/`vv.`; `volume`, `vol.`/`vols.`; ¶/¶¶; §/§§. If no locator term is used, "page" is assumed.

In complex cases, you can force something to be treated as a locator by enclosing it in curly braces or prevent parsing the suffix as locator by prepending curly braces:

```
[@smith{ii, A, D-Z}, with a suffix]
[@smith, {pp. iv, vi–xi, (xv)-(xvii)} with suffix here]
[@smith{}, 99 years later]
```

A minus sign (`-`) before the `@` will suppress mention of the author in the citation. This can be useful when the author is already mentioned in the text:

```
Smith says blah [-@smith04].
```

You can also write an author-in-text citation, by omitting the square brackets:

```
@smith04 says blah.

@smith04 [p. 33] says blah.
```

This will cause the author's name to be rendered, followed by the bibliographical details. Use this form when you want to make the citation the subject of a sentence.

When you are using a note style, it is usually better to let citeproc create the footnotes from citations rather than writing an explicit note. If you do write an explicit note that contains a citation, note that normal citations will be put in parentheses, while author-in-text citations will not. For this reason, it is sometimes preferable to use the author-in-text style inside notes when using a note style.

Non-default extensions

The following Markdown syntax extensions are not enabled by default in pandoc, but may be enabled by adding `+EXTENSION` to the format name, where `EXTENSION` is the name of the extension. Thus, for example, `markdown+hard_line_breaks` is Markdown with hard line breaks.

Extension: `rebase_relative_paths`

Rewrite relative paths for Markdown links and images, depending on the path of the file containing the link or image link. For each link or image, pandoc will compute the directory of the containing file, relative to the working directory, and prepend the resulting path to the link or image path.

The use of this extension is best understood by example. Suppose you have a subdirectory for each chapter of a book, `chap1`, `chap2`, `chap3`. Each contains a file `text.md` and a number of images used in the chapter. You would like to have `![image](spider.jpg)` in `chap1/text.md` refer to `chap1/spider.jpg` and `![image](spider.jpg)` in `chap2/text.md` refer to `chap2/spider.jpg`. To do this, use

```
pandoc chap*/*.md -f markdown+rebase_relative_paths
```

Without this extension, you would have to use `![image](chap1/spider.jpg)` in `chap1/text.md` and `![image](chap2/spider.jpg)` in `chap2/text.md`. Links with relative paths will be rewritten in the same way as images.

Absolute paths and URLs are not changed. Neither are empty paths or paths consisting entirely of a fragment, e.g., `#foo`.

Note that relative paths in reference links and images will be rewritten relative to the file containing the link reference definition, not the file containing the reference link or image itself, if these differ.

Extension: `mark`

To highlight out a section of text, begin and end it with with ==. Thus, for example,

```
This ==is deleted text.==
```

Extension: `attributes`

Allows attributes to be attached to any inline or block-level element when parsing `commonmark`. The syntax for the attributes is the same as that used in `header_attributes`.

- Attributes that occur immediately after an inline element affect that element. If they follow a space, then they belong to the space. (Hence, this option subsumes `inline_code_attributes` and `link_attributes`.)
- Attributes that occur immediately before a block element, on a line by themselves, affect that element.
- Consecutive attribute specifiers may be used, either for blocks or for inlines. Their attributes will be combined.
- Attributes that occur at the end of the text of a Setext or ATX heading (separated by whitespace from the text) affect the heading element. (Hence, this option subsumes `header_attributes`.)
- Attributes that occur after the opening fence in a fenced code block affect the code block element. (Hence, this option subsumes `fenced_code_attributes`.)
- Attributes that occur at the end of a reference link definition affect links that refer to that definition.

Note that pandoc's AST does not currently allow attributes to be attached to arbitrary elements. Hence a Span or Div container will be added if needed.

Extension: `old_dashes`

Selects the pandoc <= 1.8.2.1 behavior for parsing smart dashes: - before a numeral is an en-dash, and -- is an em-dash. This option only has an effect if `smart` is enabled. It is selected automatically for `textile` input.

Extension: `angle_brackets_escapable`

Allow < and > to be backslash-escaped, as they can be in GitHub flavored Markdown but not original Markdown. This is implied by pandoc's default `all_symbols_escapable`.

Extension: `lists_without_preceding_blankline`

Allow a list to occur right after a paragraph, with no intervening blank space.

Extension: `four_space_rule`

Selects the pandoc <= 2.0 behavior for parsing lists, so that four spaces indent are needed for list item continuation paragraphs.

Extension: `spaced_reference_links`

Allow whitespace between the two components of a reference link, for example,

```
[foo] [bar].
```

Extension: `hard_line_breaks`

Causes all newlines within a paragraph to be interpreted as hard line breaks instead of spaces.

Extension: `ignore_line_breaks`

Causes newlines within a paragraph to be ignored, rather than being treated as spaces or as hard line breaks. This option is intended for use with East Asian languages where spaces are not used between words, but text is divided into lines for readability.

Extension: `east_asian_line_breaks`

Causes newlines within a paragraph to be ignored, rather than being treated as spaces or as hard line breaks, when they occur between two East Asian wide characters. This is a better choice than `ignore_line_breaks` for texts that include a mix of East Asian wide characters and other characters.

Extension: `emoji`

Parses textual emojis like `:smile:` as Unicode emoticons.

Extension: `tex_math_gfm`

Supports two GitHub-specific formats for math. Inline math: `$`e=mc^2`$`.

Display math:

```
``` math
e=mc^2
```
```

Extension: `tex_math_single_backslash`

Causes anything between `\(` and `\)` to be interpreted as inline TeX math, and anything between `\[` and `\]` to be interpreted as display TeX math. Note: a drawback of this extension is that it precludes escaping `(` and `[`.

Extension: `tex_math_double_backslash`

Causes anything between `\\(` and `\\)` to be interpreted as inline TeX math, and anything between `\\[` and `\\]` to be interpreted as display TeX math.

Extension: `markdown_attribute`

By default, pandoc interprets material inside block-level tags as Markdown. This extension changes the behavior so that Markdown is only parsed inside block-level tags if the tags have the attribute `markdown=1`.

Extension: `mmd_title_block`

Enables a MultiMarkdown style title block at the top of the document, for example:

```
Title:   My title
Author:  John Doe
Date:    September 1, 2008
Comment: This is a sample mmd title block, with
         a field spanning multiple lines.
```

See the MultiMarkdown documentation for details. If `pandoc_title_block` or `yaml_metadata_block` is enabled, it will take precedence over `mmd_title_block`.

Extension: `abbreviations`

Parses PHP Markdown Extra abbreviation keys, like

```
*[HTML]: Hypertext Markup Language
```

Note that the pandoc document model does not support abbreviations, so if this extension is enabled, abbreviation keys are simply skipped (as opposed to being parsed as paragraphs).

Extension: `alerts`

Supports GitHub-style Markdown alerts, like

```
> [!TIP]
> Helpful advice for doing things better or more easily.
```

Note: This extension currently only works with commonmark: `commonmark`, `gfm`, `commonmark_x`.

Extension: `autolink_bare_uris`

Makes all absolute URIs into links, even when not surrounded by pointy braces `<...>`.

Extension: `mmd_link_attributes`

Parses MultiMarkdown-style key-value attributes on link and image references. This extension should not be confused with the `link_attributes` extension.

```
This is a reference ![image][ref] with MultiMarkdown attributes.

[ref]: https://path.to/image "Image title" width=20px height=30px
       id=myId class="myClass1 myClass2"
```

Extension: `mmd_header_identifiers`

Parses MultiMarkdown-style heading identifiers (in square brackets, after the heading but before any trailing #s in an ATX heading).

Extension: `gutenberg`

Use Project Gutenberg conventions for `plain` output: all-caps for strong emphasis, surround by underscores for regular emphasis, add extra blank space around headings.

Extension: `sourcepos`

Include source position attributes when parsing `commonmark`. For elements that accept attributes, a `data-pos` attribute is added; other elements are placed in a surrounding Div or Span element with a `data-pos` attribute.

Extension: `short_subsuperscripts`

Parse MultiMarkdown-style subscripts and superscripts, which start with a '~' or '^' character, respectively, and include the alphanumeric sequence that follows. For example:

```
x^2 = 4
```

or

```
Oxygen is O~2.
```

Extension: `wikilinks_title_after_pipe`

Pandoc supports multiple Markdown wikilink syntaxes, regardless of whether the title is before or after the pipe.

Using `--from=markdown+wikilinks_title_after_pipe` results in

```
[[URL|title]]
```

while using `--from=markdown+wikilinks_title_before_pipe` results in

```
[[title|URL]]
```

Markdown variants

In addition to pandoc's extended Markdown, the following Markdown variants are supported:

- `markdown_phpextra` (PHP Markdown Extra)
- `markdown_github` (deprecated GitHub-Flavored Markdown)
- `markdown_mmd` (MultiMarkdown)
- `markdown_strict` (Markdown.pl)
- `commonmark` (CommonMark)
- `gfm` (Github-Flavored Markdown)
- `commonmark_x` (CommonMark with many pandoc extensions)

To see which extensions are supported for a given format, and which are enabled by default, you can use the command

```
pandoc --list-extensions=FORMAT
```

where `FORMAT` is replaced with the name of the format.

Note that the list of extensions for `commonmark`, `gfm`, and `commonmark_x` are defined relative to default commonmark. So, for example, `backtick_code_blocks` does not appear as an extension, since it is enabled by default and cannot be disabled.